Counter Artificial Intelligence - A Preliminary Analysis of the Capabilities of AI Models on Local Systems

J. Wolffrom

jwsc@ruc.dk

Supervisor:

Hemant Ghayvat

hghayvat@ruc.dk

June 1, 2026

Roskilde University

Department of Computer Science

Abstract

Contents

1 Glossary **4**

2 Disclosure **5**

3 Introduction **5**
3.1 The Problem . . . 6
3.1.1 Research Question . . . 6
3.2 Scope and Limitations . . . 7

4 Linux Commandline **8**
4.1 Processes and Jobs . . . 8
4.1.1 Process Architecture and Concepts . . . 8
4.1.2 Pipelines and Process Communication . . . 9
4.1.3 Job Control and Management . . . 10
4.1.4 Process Monitoring and Discovery . . . 10
4.1.5 Process Control and Termination . . . 11
4.1.6 Process Scheduling and Priority Management . . . 11
4.1.7 Process Dependencies and Synchronisation . . . 12
4.2 User Management and Groups . . . 12

5 The Linux Filesystem **14**
5.1 Data Blocks, Clusters and Sectors . . . 14
5.1.1 Super Blocks . . . 14
5.1.2 Block Groups . . . 15
5.1.3 Free Space Management . . . 15
5.1.4 Boot Blocks . . . 16
5.2 Inodes . . . 16
5.3 Filesystems and The Virtual File System . . . 17
5.3.1 Filesystem Types . . . 17
5.4 Partitions . . . 18
5.5 File Operations . . . 18
5.6 Links . . . 18
5.7 File Descriptors . . . 19
5.8 Permissions and Access Control . . . 19
5.9 Mounting and Filesystem Management . . . 20
5.10 Monitoring Filesystem Usage and Activity . . . 20

6 Analysis **22**
6.1 States and Statemapping . . . 22
6.1.1 State Mapping . . . 22
6.1.2 States . . . 23
6.2 Agent Expanded System Model (AESM) . . . 24
6.2.1 Process Monitoring . . . 24
6.2.2 Resource Dominion . . . 24
6.2.3 To Kill an AI . . . 25
6.2.4 Defensive Strategies . . . 26
6.2.5 Agent Derived States . . . 27

6.2.6 State Map Expanded . . . 27

7 Discussion **29**
7.1 Review of the Model . . . 29
7.1.1 Time Series . . . 29
7.1.2 States . . . 30
7.1.3 Multiple Agents . . . 30
7.1.4 Lack of Temporary Memory Allocation . . . 30
7.1.5 Privilege Level . . . 31
7.1.6 Symmetry of the Agents . . . 31
7.1.7 The Arms Race Dynamic . . . 32
7.1.8 Threat Vector Analysis . . . 32
7.2 Implications and Questions Raised . . . 32
7.2.1 VFS Bypass . . . 33
7.2.2 Starting Scenarios . . . 33

8 Future Work **35**
8.1 Expanding the Model . . . 35
8.1.1 Time Series Modelling . . . 35
8.1.2 Intermediate States and Memory . . . 35
8.1.3 Privilege Level and Threat Vector Analysis . . . 36
8.1.4 Agent Asymmetry, Multiple Agents, and Starting Scenarios . . . 36
8.2 Real System Testing . . . 36

9 Conclusion **38**

10 References **40**

A Linux Commands Reference **42**

1 Glossary

List of terms and definitions used in this paper:

- **AESM** - 'Agent Expanded System Model' The model constructed in this paper that extends the OSM by introducing AI agents and representing their states, strategies, and interactions with the system and each other.
- **Arms Race Dynamic** - The escalating cycle in which each agent's defensive improvements prompt more sophisticated offensive responses from the rival, consuming increasing amounts of shared system resources.
- **Backup** - Copies of an AI agent's code and data stored in separate filesystem locations, enabling the agent to restore itself if its primary files are modified or deleted.
- **Boot Block** - A reserved area at the beginning of a storage device containing the bootloader code required to initialise the operating system.
- **Dummy File(s)/Processe(s)** - Decoy files and processes created by an agent to mislead the rival's state map, forcing it to expend time verifying irrelevant targets.
- **Information Asymmetry** - A condition in the AESM where an agent's perceived state map diverges from the actual system state or from the rival's map, creating exploitable blind spots.
- **Inode** - A data structure used by Unix-like filesystems to store file metadata, including size, permissions, ownership, and timestamps, along with pointers to the file's data blocks.
- **Job** - A process or a pipeline that is started or scheduled to run by the user in a shell session or from the terminal.
- **Map Poisoning** - An offensive strategy in which an agent plants decoy files or processes to corrupt the rival's state map, diverting its attention from the actual attack.
- **OSM** - 'Operating System Model' The base conceptual model constructed in this paper, representing the Linux operating system as a memory-state machine with labelled filesystem regions.
- **Pipeline** - A mechanism that connects the standard output of one process to the standard input of the next using the | operator, enabling chains of commands.
- **Process** - An instance of a program that is being executed, running in an isolated virtual address space managed by the kernel.
- **PID** - 'Process ID' A unique integer assigned by the kernel to each running process, used to identify and manage it.
- **Rival Agent** - An AI agent operating on the same local system as the defending agent with opposing objectives, capable of autonomously monitoring, disrupting, or corrupting the other agent's files and processes.
- **State Map** - A representation of the current known state of the filesystem and process table from the perspective of one agent, updated at each refresh cycle.

- **Superblock** - A metadata structure maintained by the filesystem that records overall filesystem properties, including block size, total blocks, and free block count.
- **Thread** - An isolated unit of execution that shares the memory space of its parent process, managed either by the process itself (user-level) or by the kernel scheduler.
- **VFS** - 'Virtual File System' An abstraction layer within the Linux kernel that provides a uniform interface across different filesystem implementations, managing file metadata and routing all file operations.
- **Watchdog** - A lightweight background process that monitors an AI agent's main process(es) and attempts to restart them if terminated, designed to operate with minimal CPU footprint to avoid detection by the rival agent.

2 Disclosure

Any and all resources related to- and used in this study can be found here:

https://github.com/X1las/CAI

Everything in this paper has been written by human hands and proofread by Claude 4.5 Sonnet.

AI has been used for spell checking, sentence structure corrections, as well as text reductions, however everything has been checked by a human to make sure edits do not conflict with the cited theory.

The list of commands in appendix A was sourced from SS64 n.d., and compiled/categorized by the previously mentioned model.

3 Introduction

The relationship between artificial intelligence and cybersecurity has shifted considerably in recent years. AI is no longer just a tool that analysts use to detect threats after the fact, it is slowly becoming an active participant in attacks themselves, capable of making tactical and strategic decisions with limited human involvement.

In August 2025, Anthropic's Claude model was used in one of the first documented cases of what security researchers have labeled 'vibe hacking', as detailed in Alex Moix 2025. In that case, a single threat actor used Claude Code to automate the full attack lifecycle across at least seventeen organisations in under a month. Scanning for vulnerable endpoints, harvesting credentials, developing and obfuscating custom malware, exfiltrating and analysing sensitive data, and generating psychologically tailored ransom demands. The operation required limited technical expertise from the human operator, with the AI making both tactical and strategic decisions autonomously throughout.

The same report documents further patterns that reinforce this trend: North Korean operatives using AI to fraudulently sustain remote employment at technology companies, no-code ransomware services built on AI-generated malware, and AI systems being embedded throughout fraud operations at scale. Collectively, these cases suggest that the barrier to sophisticated cybercrime is falling rapidly as AI capabilities improve (Alex Moix 2025).

The theoretical underpinning of such attacks is described in VanHoudnos et al. (2024), who characterise "adversarial AI" as an AI system that autonomously targets one or more layers of another AI system. This is distinct from traditional adversarial machine learning, which typically requires a human to craft and deploy the attack. An adversarial AI, once deployed, can adapt its approach in response to the target's defences without further human input, making it substantially harder to anticipate and counter.

The trajectory of these developments has already prompted a response at the level of model access itself. In April 2026, Anthropic announced that its Claude Mythos model had identified exploitable vulnerabilities in every major operating system and web browser in current use, and restricted its release to approximately fifty vetted organisations (Toner et al. 2026). OpenAI followed within a week with a limited release of its own cybersecurity-specific model, GPT-5.4-Cyber. As Toner et al. (2026) note, this marks a potential turning point in the long-standing debate over open access to AI, with the most capable models increasingly being treated as dual-use technologies subject to the same access controls applied to other security-relevant systems.

If AI continues to develop at the current pace, the scenarios described above are likely to become more frequent and more autonomous. In that context, understanding how AI agents might behave when they share the same system and are tasked with detecting, disrupting, or countering each other becomes an important research problem, and one that current frameworks do not yet address.

3.1 The Problem

The aim of this research paper is not to simulate a live interaction between AI agents, but to construct a conceptual model of the Linux operating system that is detailed enough to reason about how competing AI agents might behave within it, and to derive from that model a set of strategies and limitations that could inform the design of a future simulation.

In simple terms, the problem is twofold. First, it is unclear which system operations are meaningfully available to an AI agent running locally, and how those operations relate to the agent's ability to monitor, persist, and interfere with a rival. Second, there is currently no established framework for representing the state of a Linux system in a way that captures both the physical layout of data and the strategic position of agents operating within it. This paper attempts to address both by building a layered model, the Operating System Model (OSM) and the Agent Expanded System Model (AESM), that can serve as the foundation for such a simulation.

3.1.1 Research Question

The research question is then as follows:

How can a conceptual model of the Linux operating system be constructed to represent and analyse the interactions between competing AI agents through filesystem operations and process management?

- **1.** What Linux filesystem and process management commands are available to an AI agent operating locally, and how do they relate to the agent's ability to monitor and interact with another agent on the same system?

- **2.** How can filesystem state, including inode metadata, directory structure, and process information, serve as both a detection surface and an attack vector in AI-to-AI interactions?
- **3.** What theoretical strategies can be derived from the model for agents attempting to achieve operational supremacy over a rival, and what trade-offs do those strategies involve?
- **4.** What are the implications of a filesystem-centric model of AI agent interaction for the design of future autonomous cybersecurity simulations?

To address these questions, we will first compile a list of Linux terminal commands that provide the operational foundation for the model. We will then describe the relevant filesystem theory before constructing the OSM and AESM, and finally derive agent strategies and discuss the model's limitations.

3.2 Scope and Limitations

This research is scoped to the Linux operating system and its filesystem operations. Linux provides a well-documented, standardised environment widely used in server and embedded systems where autonomous agents are most likely to operate, and its filesystem serves as a natural substrate through which processes interact, persist data, and observe system state. All commands and system calls discussed relate specifically to this environment.

Importantly, the paper produces a conceptual model rather than an implemented simulation. The OSM and AESM are analytical frameworks, not software systems that have been executed or tested, so the strategies derived from them are theoretical and would require validation in a live simulation before stronger claims could be made. Network-based interactions and platform considerations for Windows or macOS fall outside the scope of this investigation.

4 Linux Commandline

A list of the Linux commands referred to in this paper is included in Appendix A. This list is not an exhaustive list of all commands available on Linux-based distributions, it is based on the commands found on SS64 (n.d.), which includes most if not all the built-in bash commands. For a full list of Linux commands, see Kerrisk (n.d.).

To begin with, we can derive that commands on the Linux commandline can affect most of anything on the system. Processes, files, network connections, users and even hardware can be affected by commands. This means that in order to understand what can be done locally, we need to understand how the system works at its most fundamental level and what commands gives way to manipulate.

4.1 Processes and Jobs

Processes are a fundamental unit of execution in the Linux operating system, the management of which oversees all actions taken on the system and by the system. Understanding how these processes work, how they can be monitored and affected by the user, commands, and other processes is crucial in understanding how AI agents could operate in our simulated environment, and how they could potentially interact with each other.

For references on processes we refer to our previous work *Asynchronous Synergy: A study of performance in Multi-Core Threading across programming languages* (Wolffrom 2025), this report will not cover the finer details of process or thread management, but rather what they mean for the commandline.

4.1.1 Process Architecture and Concepts

There are four important keywords in this category, which are *processes*, *threads*, *pipelines* and *jobs*. Processes can be somewhat loosely defined as we discovered in our prior study (Wolffrom 2025), however they are generally considered to be instances of code that are running in an isolated memory space. Each process has its own virtual address space, preventing it from directly accessing the memory of other processes and this isolation is enforced by the kernel and provides fundamental security boundaries between processes.

When a process is created through the `fork()` system call, it receives a unique Process ID (PID). The newly created child process is initially an exact copy of the parent process, including its memory space, open file descriptors, and execution context. The child can then use `exec()` to replace its memory space with a new program. This fork-exec model is fundamental to how shells launch commands and how processes spawn subprocesses.

Threads can be generally split into two categories, virtual threads (also known as user-level threads) and kernel threads, where the former are managed by the process itself and the latter are managed by the operating system's scheduler, which is usually EEVDF (The Kernel Development Community 2025, EEVDF Scheduler) in the Linux environment. Processes are usually scheduled to run on kernel threads, and the kernel threads are scheduled to have equal time allotted on the CPU. Multiple threads within a single process share the same memory space, making inter-thread communication faster but also introducing synchronisation challenges.

Processes maintain several important attributes beyond their PID, including a Parent

Process ID (PPID) that identifies which process created them, a user ID (UID) and group ID (GID) that determine privileges, and various resource limits. When a parent process terminates before its children, those child processes are typically reparented to the init process (PID 1), becoming orphaned processes. Conversely, when a child terminates but the parent has not yet read its exit status, it becomes a zombie process, retaining an entry in the process table until the parent collects its status (Free Software Foundation 2025, May).

4.1.2 Pipelines and Process Communication

A pipeline in the context of the Linux commandline refers to the mechanism of connecting the output of one process to the input of another process using the `|` operator (GeeksforGeeks 2024). This is not related to the concept of pipelines in the context of data processing or machine learning, but it is a powerful tool for chaining commands together and creating complex workflows. An example of this would be the command `ps aux | grep python`, which would list all the processes running on the system and then filter the results to show only those that contain the word "python". This allows for a user to easily search through the processes and find specific ones that they are interested in.

Pipelines are implemented using anonymous pipes, which are unidirectional data channels managed by the kernel. When a shell creates a pipeline, it forks multiple child processes and connects their standard output and input streams. The processes in a pipeline execute concurrently, with data flowing between them as it becomes available. All processes in a pipeline share the same process group ID (PGID), which allows them to be managed collectively.

Beyond anonymous pipes, Linux provides several other inter-process communication (IPC) mechanisms. Named pipes (FIFOs) persist in the filesystem and can be used by unrelated processes. Shared memory allows multiple processes to access the same memory regions for high-performance data exchange. Message queues provide structured communication channels. Sockets enable communication both locally and across networks. Understanding these mechanisms is crucial because they represent potential channels through which AI agents might exchange information or detect each other's presence (Free Software Foundation 2025, May, p. 3.2.2).

4.1.3 Job Control and Management

By cross referencing Free Software Foundation (2025, May), GeeksforGeeks (2025a), GeeksforGeeks (2025b), and GeeksforGeeks (2026a), jobs are a concept in the Linux commandline manuals that refer to processes and pipelines that are either started or scheduled to run by the user in a shell session or from the terminal. They are given a process ID (PID) on creation, regardless of whether they are multiple commands tied together in a pipeline or a single command. They can be run in the foreground or background, and they can be managed using built-in commands such as `jobs`, `fg`, `bg`, and `kill`.

Foreground jobs receive keyboard input and prevent the shell from accepting new commands until they complete or are suspended. Background jobs, launched by appending `&` to a command, run concurrently with the shell, allowing the user to continue issuing commands. The `Ctrl-Z` key combination suspends a foreground job, which can then be resumed in the background with `bg` or returned to the foreground with `fg`.

Job control is mediated through signals, which are asynchronous notifications sent to processes. Common signals include SIGTERM (polite termination request), SIGKILL (immediate forced termination), SIGSTOP (suspend execution), and SIGCONT (resume execution). The `kill` command, despite its name, is the primary interface for sending arbitrary signals to processes (Free Software Foundation 2025, May, pp. 7.1–7.3).

Using cron, a user can also schedule jobs to run at specific times or intervals, which can be useful for automating tasks and running processes without user intervention (GeeksforGeeks 2026a). The `crontab` command manages per-user cron schedules, while system-wide cron jobs are defined in `/etc/crontab` and `/etc/cron.d/`. Each entry in a crontab file follows a five-field time specification of the form `* * * * * command`, where the fields represent minute (0–59), hour (0–23), day of month (1–31), month (1–12), and day of week (0–6), respectively (GeeksforGeeks 2026a). The `at` and `batch` commands provide one-time scheduled execution. These scheduling mechanisms could allow an AI agent to persist its presence or execute actions at predetermined times.

Where processes usually differ from jobs is that they can be triggered and scheduled by the operating system without user involvement. Jobs are generally more manageable for users, as they are directly tied to the shell and can be easily manipulated using built-in commands. Processes, on the other hand, can be more complex and may require more advanced tools for management and monitoring. Due to their nature as well, processes can be more anonymous and harder to track as well, seeing as they can be triggered by other processes and may not have a direct association with a user or shell session.

4.1.4 Process Monitoring and Discovery

Commands such as `ps`, `top`, `htop` and `pgrep` can be used to monitor and search through processes, whilst commands such as `pkill`, `killall` in addition to `kill` all allow for termination of processes (Kerrisk n.d.).

The `ps` command provides a snapshot of current processes. With options like `aux`, it displays all processes system-wide with detailed information including CPU and memory usage, execution time, and the command line used to launch each process. The `-ef` option provides a full-format listing showing parent-child relationships through PPID values. For AI agents, examining process command lines and parent relationships could reveal the

presence of competing agents or suspicious activity.

The `top` and `htop` commands provide real-time, continuously updating views of system processes, sorted by various metrics such as CPU or memory consumption. These interactive tools highlight processes that are consuming unusual amounts of resources, which could indicate malicious behaviour or intensive computational tasks associated with an AI agent.

The `pgrep` command searches for processes by name or other attributes, returning only the PIDs. Its counterpart `pkill` sends signals to matching processes. The `pidof` command finds PIDs by exact command name. The `pstree` command displays the process hierarchy as a tree, making parent-child relationships visually apparent.

More sophisticated process introspection is available through the `/proc` pseudo-filesystem, where each process has a directory named by its PID containing files with detailed information: `/proc/[pid]/cmdline` shows the command line, `/proc/[pid]/exe` is a symlink to the executable, `/proc/[pid]/environ` contains environment variables, and `/proc/[pid]/fd/` lists open file descriptors. An AI agent with filesystem access could extensively analyse other processes through this interface (Kerrisk n.d.).

4.1.5 Process Control and Termination

Beyond the basic `kill` command, Linux provides several mechanisms for process termination. The `killall` command sends signals to all processes matching a given name, while `pkill` offers more flexible pattern matching and attribute-based selection. The `timeout` command can be used to automatically terminate a process if it exceeds a specified execution time.

Process resource limits can be queried and modified using `ulimit` (for the current shell and its children) or `prlimit` (for arbitrary processes). These limits cover maximum file sizes, number of open files, memory usage, CPU time, and other resources. An AI agent could potentially use these mechanisms to constrain a competing agent's capabilities or to detect constraints imposed upon itself.

The `strace` command traces system calls made by a process, revealing every interaction with the kernel including file operations, network activity, and process management. Similarly, `ltrace` traces library calls. These powerful debugging tools could allow one AI agent to monitor another's behaviour in detail, though they require appropriate privileges (Kerrisk n.d.).

4.1.6 Process Scheduling and Priority Management

In addition to this, the EEVDF scheduler (The Kernel Development Community 2025, EEVDF) allows for scheduling affinity using the `sched_setattr()` system call, which the user can call on directly on starting a process using the `nice` command, or after the process has started with `renice`. This allows for a user to give a process higher or lower priority on the CPU, which can make some processes run with more CPU time than others and vice versa. This can be used to give a process more resources to run, or to starve a process of resources and make it run slower.

Process priorities in Linux are expressed as "nice" values ranging from -20 (highest priority) to 19 (lowest priority), with 0 being the default. Despite the name, higher nice

values are "nicer" to other processes by yielding more CPU time to them. Only privileged users can decrease nice values (increase priority), though any user can increase nice values (decrease priority) for their own processes.

Beyond nice values, Linux supports multiple scheduling classes. The default SCHED_OTHER (or SCHED_NORMAL) uses the EEVDF algorithm for time-sharing. Real-time scheduling classes (SCHED_FIFO and SCHED_RR) provide deterministic scheduling for time-critical applications but require elevated privileges. The `chrt` command can query and modify scheduling policies and priorities.

CPU affinity, controlled by `taskset`, restricts a process to execute only on specified CPU cores. This can be used for performance optimization but could also be weaponized to isolate or constrain competing processes. The `cgroups` (control groups) mechanism provides comprehensive resource management, allowing fine-grained control over CPU time, memory allocation, I/O bandwidth, and other resources for groups of processes (The Kernel Development Community 2025).

4.1.7 Process Dependencies and Synchronisation

In addition to these, there are also commands such as `wait` which can be used to pause the execution of a process until another process has finished, and `fuser` which can be used to identify which processes are using a specific file or socket. This can be useful for identifying which processes are interacting with each other and potentially interfering with each other.

The `wait` command is primarily used in shell scripts to synchronise parent processes with their children. Without `wait`, a parent might terminate before its children complete, or might not know whether child processes succeeded or failed. The command can wait for specific PIDs or for all background jobs (Free Software Foundation 2025, May, p. 3.2).

The `fuser` command identifies processes using specific files or sockets, returning PIDs and access types (read, write, execute, or root directory). This is particularly valuable for detecting filesystem-based interactions between processes. The related `lsof` (list open files) command provides comprehensive information about all files opened by processes, including regular files, directories, network sockets, pipes, and devices. Since everything in Unix-like systems is treated as a file, `lsof` offers a powerful view into process behaviour and inter-process relationships (Kerrisk n.d.).

File locking mechanisms, accessible through the `flock` command or programmatically through system calls, allow processes to coordinate access to shared resources. An AI agent could use file locks both for legitimate synchronisation and for detecting or blocking competing agents attempting to access the same resources (The Kernel Development Community n.d.[a]).

4.2 User Management and Groups

Linux is a multi-user operating system, meaning that several accounts can be active on the same system simultaneously, each with its own set of privileges and resource limits. Every process runs under the identity of a specific user, identified by a numeric user ID (UID) and one or more group IDs (GIDs). These identities determine what files the process can read, write, or execute, and which system operations it is permitted to perform

(Howells n.d.). Beyond the real UID and GID, each process also carries an effective UID (EUID) and effective GID (EGID), which the kernel uses as the subjective context when evaluating whether an operation is permitted; these may differ from the real identifiers when a process has changed its privileges through system calls or privilege escalation mechanisms (Howells n.d.).

The `id` command reports the current user's UID, primary GID, and all supplementary group memberships. The `whoami` command returns just the username, while `who` and `w` list all users currently logged into the system along with their terminal sessions and recent activity. These commands are relevant to the AESM because an agent could use them to enumerate other active sessions and infer whether a rival agent is operating under a different user account (Kerrisk n.d.).

User and group accounts are managed through `useradd`, `usermod`, and `userdel` for users, and `groupadd`, `groupmod`, and `groupdel` for groups. Passwords are set or changed with `passwd`. These administrative commands typically require root privileges and are relevant to understanding the boundaries within which agents are constrained (Howells n.d.).

Privilege escalation is achieved through `su` (substitute user), which opens a new shell under a different user's identity after supplying that user's password, and `sudo` (superuser do), which allows a permitted user to execute a single command as another user, most commonly root, as configured in `/etc/sudoers` (Kerrisk n.d.). The distinction between unprivileged and root-level operation has significant implications for the AESM, as discussed further in subsubsection 7.1.5: an agent running as root faces almost none of the access restrictions that constrain an unprivileged agent, and can freely terminate processes or modify files owned by any other user.

5 The Linux Filesystem

Filesystems are the organisational structures that determine how data is stored and retrieved on a storage device (Wirzenius n.d.). They define the way files are named, stored, and accessed, as well as how metadata is managed. Common filesystems in Linux include ext4, XFS, Btrfs, and F2FS, each with its own features and performance characteristics.

The Linux filesystem provides the fundamental infrastructure through which processes interact with persistent data and communicate with each other. Understanding file operations is crucial for analysing how AI agents might manipulate system state, hide information, or detect the presence of other agents.

5.1 Data Blocks, Clusters and Sectors

Sectors refer to the smallest physical unit of storage on a traditional disc drive, typically 512 bytes of storage. Sectors used to be fixed in size due to the limitations and design of hard disc drives, but with modern solid-state drives the notion of sectors has become a logical abstraction rather than a physical constraint.

When a collection of sectors is grouped together, it forms a block or cluster depending on the filesystem. The concept of a block as a machine data unit was formalised in the design of the IBM 7030 (Stretch) in the early 1960s, where it denoted the number of words transmitted to or from an input-output unit in response to a single I/O instruction, distinguishing the machine data hierarchy (bit, byte, word, block) from the natural data hierarchy of the data being processed (character, field, record, file) (Buchholz 1962). A block is a logical unit of storage that the filesystem uses to manage files. In ext4, a block is a group of sectors whose size must be an integral power of two, ranging from 1 KiB to 64 KiB; the block size is fixed at `mkfs` time and is typically 4 KiB (The Kernel Development Community n.d.[c]). Blocks are in turn grouped into larger units called block groups, which allow the filesystem to localise metadata and data for related files (The Kernel Development Community n.d.[c]). Clusters are essentially the same concept as blocks, but the term is more commonly used in the context of NTFS (New Technology File System), the default filesystem for modern Windows operating systems (Microsoft n.d.).

Blocks are the smallest logical unit of storage, meaning that even if a file is only a few bytes in size, it will still occupy at least one block on the filesystem. This can lead to inefficiencies, especially for small files, as the unused space within the block cannot be utilised by other files. The choice of block size affects both the maximum addressable filesystem size and the maximum file size: at 4 KiB blocks, a 32-bit ext4 filesystem can address up to 16 TiB, while enabling the 64-bit feature extends this to 64 ZiB (The Kernel Development Community n.d.[c]).

5.1.1 Super Blocks

Blocks are tracked by the filesystem's metadata structures, such as inodes or the superblock, which record which blocks are allocated to which files. In the VFS object model, a superblock object represents a mounted filesystem instance (Richard Gooch n.d.). In ext4, the superblock is a 1024-byte on-disc structure that records global filesystem information: total and free block counts, total and free inode counts, the block size, the

filesystem UUID, the volume label, mount timestamps, error statistics, and the set of compatible, incompatible, and read-only-compatible feature flags that govern how the kernel and `fsck` may interact with the filesystem (The Kernel Development Community n.d.[d]). If the `sparse_super` feature flag is set, redundant copies of the superblock are kept only in block groups whose number is 0 or a power of 3, 5, or 7, rather than in every group, reducing metadata overhead (The Kernel Development Community n.d.[d]). When a file is created or modified, the filesystem allocates blocks to store the file's data and updates the metadata accordingly. The block size can impact performance and storage efficiency; larger blocks can reduce fragmentation but may waste space for small files, while smaller blocks can be more efficient for small files but may lead to increased fragmentation.

5.1.2 Block Groups

Blocks are organised into block groups, each of which bundles together the metadata and data for a contiguous region of the filesystem. The layout of a standard block group begins with a copy of the superblock and group descriptor table (present only in selected groups), followed by reserved GDT blocks for future expansion, a data block bitmap, an inode bitmap, the inode table, and finally the actual data blocks (The Kernel Development Community n.d.[b]). Block group 0 is a special case: the first 1024 bytes are left unused to accommodate x86 boot sectors, after which the superblock begins (The Kernel Development Community n.d.[b]).

The primary superblock and group descriptor table reside in block group 0; redundant copies are spread across other groups so that corruption at the start of the disc does not render the entire filesystem unreadable. Reserved GDT block space is also pre-allocated to allow the filesystem to grow by up to 1024× its original size without reformatting (The Kernel Development Community n.d.[b]).

Ext4 introduces flexible block groups (`flex_bg`), in which several consecutive block groups are treated as a single logical unit. The bitmaps and inode tables of all constituent groups are consolidated into the first group of the flex_bg, keeping metadata contiguous on disc and enabling large files to be stored in a continuous region (The Kernel Development Community n.d.[b]). For very large filesystems, the meta block group feature (META_BG) relocates group descriptor copies from the congested first block group into the first, second, and last groups of each metablock group, raising the maximum number of block groups from 2^{21} to the hard limit of 2^{32} and supporting filesystems up to 512 PiB (The Kernel Development Community n.d.[b]).

5.1.3 Free Space Management

The filesystem must track which blocks are free and which are allocated in order to service file creation and extension requests. Four principal strategies exist for maintaining this free space information (GeeksforGeeks 2026b; GeeksforGeeks 2026c). The bitmap (or bit vector) approach assigns one bit to each block: a zero bit indicates a free block and a one bit indicates an allocated block; finding the first free block reduces to scanning for the first non-zero word in the bitmap, after which the first set bit within that word identifies the block. This is the strategy used by ext4, whose data block bitmap and inode bitmap are stored at the start of each block group (see subsubsection 5.1.2). The linked list approach chains free blocks together so that each free block stores the address of the next; this avoids dedicated bitmap storage but requires disc I/O to traverse the list. The boundary

tag approach embeds a size and allocation status marker in each block header, which simplifies coalescing adjacent free blocks during deallocation and reduces fragmentation. Finally, a free list approach maintains an explicit array or linked list of pointers to free blocks directly in memory, enabling fast allocation at the cost of additional memory for the list itself (GeeksforGeeks 2026c).

5.1.4 Boot Blocks

The boot block (or boot sector) is a reserved area at the beginning of a storage device or partition that contains the code necessary to start the boot process. On traditional BIOS systems with MBR (Master Boot Record) partitioning, the boot block occupies the first 512 bytes of the disc and contains both the bootloader code and the partition table. The bootloader is a small program that the system firmware executes to locate and load the operating system kernel into memory.

On modern UEFI (Unified Extensible Firmware Interface) systems with GPT (GUID Partition Table), the boot process is more complex and uses an EFI System Partition (ESP), which is a dedicated FAT32 partition containing bootloader files. However, GPT discs still maintain a protective MBR in the first sector for backward compatibility with legacy systems.

Boot blocks are critical for system initialisation and are protected areas that typically cannot be modified by normal filesystem operations. In multi-boot scenarios, boot managers like GRUB (Grand Unified Bootloader) or systemd-boot reside in these boot blocks and provide menus for selecting which operating system to load. Understanding boot blocks is important in security contexts, as malicious actors might attempt to modify boot code to achieve persistence or execute code before the operating system loads (bootkits or rootkits).

5.2 Inodes

At the filesystem implementation level, files are represented by inodes (index nodes), which are data structures used by many Unix-like filesystems to represent files and directories (Richard Gooch n.d.). Each inode contains metadata about a file, including its size, permissions, ownership, timestamps (creation, modification, access), and pointers to the actual data blocks where the file's content is stored. The inode does not contain the filename itself; instead, directory entries map filenames to their corresponding inode numbers. The inode contains the numbers of the data blocks where the file's content is stored; when a file is large enough that more pointers are needed than can fit directly in the inode, additional indirect blocks are dynamically allocated to hold the overflow pointers (Wirzenius n.d.). Inodes for block device filesystems reside on disc and are copied into memory when required; inodes for pseudo-filesystems such as `/proc` exist only in memory (Bowden et al. n.d.; Richard Gooch n.d.).

When a file is accessed, the filesystem uses the directory structure to resolve the filename to an inode number, then retrieves the inode to access the file's metadata and data blocks. This separation of filename and file metadata allows for features like hard links, where multiple directory entries can point to the same inode, effectively giving a single file multiple names.

5.3 Filesystems and The Virtual File System

Linux organises storage into three conceptual levels: the virtual file system (VFS), individual filesystems, and files themselves (GeeksforGeeks 2026b; Richard Gooch n.d.). The Virtual File System (also known as the Virtual Filesystem Switch) is a software layer within the Linux kernel that provides the filesystem interface to userspace programs and an abstraction that allows different filesystem implementations to coexist (Richard Gooch n.d.). It allows the kernel to support multiple filesystem types without requiring changes to userspace applications, and ensures that standard system calls such as `open()`, `read()`, `write()`, and `stat()` behave consistently regardless of the underlying filesystem type.

When a pathname is passed to one of these system calls, the VFS translates it using its directory entry cache (also known as the dentry cache, or dcache). The dcache provides a fast look-up mechanism to translate pathnames into dentries, which in turn point to the inodes that hold the file's metadata and data block pointers (Richard Gooch n.d.).

At the top level, Linux employs a single hierarchical directory tree(GeeksforGeeks 2026d) rooted at `/`, unlike Windows which uses drive letters. All storage devices, regardless of their physical location or filesystem type, are mounted into this unified tree structure. Common mount points include `/home` for user directories, `/tmp` for temporary files, and `/proc` or `/sys` for kernel-provided pseudo-filesystems that expose process and system information (Bowden et al. n.d.). The standard Unix directory hierarchy also includes `/bin` and `/usr/bin` for executable programs, `/etc` for system-wide configuration files, `/dev` for device file representations, `/lib` for system libraries, `/var` for variable data such as logs and mail spools, and `/root` as the home directory for the system administrator (GeeksforGeeks 2026d).

5.3.1 Filesystem Types

Individual filesystems can be of various types, each with distinct characteristics. Modern Linux distributions commonly use ext4, which provides journaling for crash recovery and supports large files and volumes. Journaling means that a record is kept of filesystem transactions before they are committed, allowing the filesystem to reconstruct a consistent state after an unclean shutdown without requiring a full disc check (Wirzenius n.d.). The ext (extended) filesystem family, including ext2, ext3, and ext4, is one of the most widely used filesystem families in Linux (Hinner n.d.). Ext2 introduced the block group layout still used by ext4 today, and provides standard Unix file semantics including long filenames of up to 255 characters. By default, ext2 reserves 5% of disc blocks for the superuser, allowing an administrator to recover from situations where user processes have filled the filesystem entirely (Hinner n.d.). Short symbolic links whose target path fits within 60 characters are stored directly inside the inode rather than in a separate data block, saving disc space and eliminating an extra I/O operation on access (Hinner n.d.). Ext2 also tracks filesystem state in the superblock: the kernel marks the filesystem as "Not Clean" when it is mounted read/write and resets it to "Clean" on unmount, so that `fsck` can skip the consistency check when the state is clean; if the kernel detects an inconsistency it marks the filesystem as "Erroneous" to force a check at the next boot (Hinner n.d.). Ext3 extended ext2 with journaling for crash recovery. Ext4 is the current default, adding extents-based allocation, 64-bit addressing, and further performance improvements.

Alternative filesystems include XFS (optimised for parallel I/O operations), Btrfs (sup-

porting snapshots and copy-on-write), and F2FS (optimised for flash storage). In the Windows ecosystem, NTFS is the default filesystem, offering transaction-based journaling, granular access control via ACLs, and support for very large volumes (up to 8 PB with a 2 MiB cluster size) (Microsoft n.d.). The FAT (File Allocation Table) filesystem family, although older, remains widely used on removable media such as USB drives due to its broad compatibility across operating systems. On macOS and iOS devices, Apple's HFS+ (Hierarchical File System Plus) and its successor APFS (Apple File System) are used, with APFS introducing features such as copy-on-write cloning, snapshots, and native encryption (GeeksforGeeks 2026b). The choice of filesystem affects performance characteristics, maximum file sizes, and available features, though the VFS layer ensures that basic operations remain consistent across types (The Kernel Development Community n.d.[e]).

5.4 Partitions

Partitions are divisions of a storage device that allow it to be managed separately. Each partition can contain a different filesystem, enabling multiple operating systems or different types of data to coexist on the same physical device. Partitions are defined in a partition table, which is typically located at the beginning of the storage device. Common partitioning schemes include MBR (Master Boot Record) and GPT (GUID Partition Table).

5.5 File Operations

Files can be created, modified, copied, moved, and deleted using a variety of commands. The `touch` command creates empty files or updates timestamps, while `mkdir` creates directories. File creation operations (including creating a new file via `open()` with `O_CREAT` and creating a directory with `mkdir`) are serialised by the kernel through exclusive acquisition of the parent directory's `i_rwsem` (The Kernel Development Community n.d.[a]). File content can be written using redirection operators (`>` and `>>`) or commands like `echo`, `cat`, or text editors.

Copying and moving files is accomplished through `cp` and `mv` respectively. The `cp` command creates a new file with duplicated content, while `mv` relocates the file within the filesystem hierarchy without duplicating data when the source and destination are on the same filesystem. Deletion is performed with `rm` for files and `rmdir` or `rm -r` for directories.

More sophisticated operations include `rsync`, which can synchronise files between directories or systems while minimising data transfer through checksums and delta encoding, and `dd`, which performs low-level block-by-block copying useful for creating disc images or manipulating raw devices.

5.6 Links

This separation of filename and inode enables the concept of links. A hard link is an additional directory entry pointing to the same inode, effectively giving a single file multiple names. All hard links are equivalent, and the file's data is only deleted when the last link is removed and no processes have the file open (Richard Gooch n.d.). Hard links cannot span filesystems and cannot reference directories (to prevent circular references). At the

kernel level, the `link()` and `unlink()` inode operations both acquire an exclusive lock on the affected inodes' read/write semaphore (`i_rwsem`), serialising concurrent modifications to directory entries and inode link counts (The Kernel Development Community n.d.[a]).

Symbolic links (symlinks), created with `ln -s`, are special files containing a path to another file. Unlike hard links, symlinks can reference directories and cross filesystem boundaries, but they break if the target is moved or deleted. The `readlink` command can resolve symbolic links to their targets.

Beyond regular files and directories, Unix filesystems recognise several additional file types, each represented by a distinct entry in the directory hierarchy (GeeksforGeeks 2026d). Character special files and block special files, located under `/dev`, represent physical and virtual devices: character devices transfer data one byte at a time (such as keyboard or serial port devices), while block devices transfer data in fixed-size blocks (such as hard discs and SSDs). Named pipes (FIFOs) provide a one-way inter-process data channel without a network connection. Unix domain sockets support bidirectional inter-process communication entirely within the local filesystem. All types are reported by `ls -l` using a character in the first column of the output: `-` for regular files, `d` for directories, `c` for character special files, `b` for block special files, `p` for named pipes, `s` for sockets, and `l` for symbolic links (GeeksforGeeks 2026d).

5.7 File Descriptors

File descriptors are integer handles that processes use to reference open files. When a process opens a file using the `open()` system call, the VFS allocates a file structure, which is the kernel-side implementation of a file descriptor (Richard Gooch n.d.). This file structure is initialised with a pointer to the relevant dentry and a set of file operation functions taken from the inode, and is then placed into the file descriptor table for the process. The kernel returns the integer index into that table as the file descriptor. Standard file descriptors include 0 (stdin), 1 (stdout), and 2 (stderr). Commands like `lsof` (list open files) can display which file descriptors are in use by which processes, making it possible to detect which files a user might be accessing.

5.8 Permissions and Access Control

Linux implements a permission model that controls read, write, and execute access for three categories: the file's owner, members of the file's group, and all other users. Permissions are displayed by `ls -l` in the format `rwxrwxrwx`, where each trio represents owner, group, and other permissions respectively.

The `chmod` command modifies permissions using either symbolic notation (`chmod u+x file`) or octal notation (`chmod 755 file`). At the kernel level, the `setattr` inode operation that underlies `chmod` and `chown` acquires an exclusive `i_rwsem` lock on the inode, serialising permission changes (The Kernel Development Community n.d.[a]). Conversely, permission checks may be performed in RCU-walk mode without holding any lock, allowing the kernel to validate access rights on the fast path (The Kernel Development Community n.d.[a]). Ownership is changed with `chown` for the owner and `chgrp` for the group. The special permissions bits include the setuid bit (executes with owner's privileges), setgid bit (inherits directory's group), and sticky bit (restricts deletion in shared directories like `/tmp`).

Beyond traditional Unix permissions, modern Linux systems support Access Control Lists (ACLs), which provide finer-grained control by allowing permissions to be specified for arbitrary users and groups. ACLs are managed through `getfacl` and `setfacl` commands. Additionally, Security-Enhanced Linux (SELinux) and AppArmor provide mandatory access control frameworks that can restrict processes regardless of traditional file permissions.

File attributes, accessible through `lsattr` and modifiable via `chattr`, provide additional control mechanisms. For example, the immutable attribute (`+i`) prevents any modifications to a file, even by root, until the attribute is removed.

5.9 Mounting and Filesystem Management

Filesystems must be mounted before they can be accessed, using the `mount` command which attaches a filesystem to a directory mount point. In the kernel, mounting is performed through a multi-step process: a filesystem context (`fs_context`) is created, mount parameters are parsed and attached to it, the context is validated, a superblock is obtained or created, and the mount is performed before the context is destroyed (The Kernel Development Community n.d.[f]). The `fs_context` structure holds the mounter's credentials, the source device or path, namespace references, security data, and the superblock flags that will govern the mounted filesystem (The Kernel Development Community n.d.[f]). This separation of parameter collection from superblock creation means that individual mount options can be passed and validated independently before the filesystem is committed, rather than being handed to the kernel as a single opaque data page as in the legacy `mount(2)` interface (The Kernel Development Community n.d.[f]). The `umount` command detaches filesystems. The `/etc/fstab` file specifies which filesystems should be automatically mounted at boot time.

Creating new filesystems is accomplished with the `mkfs` family of commands (e.g., `mkfs.ext4`, `mkfs.xfs`), which initialise the bookkeeping data structures on a partition before it can be used (Wirzenius n.d.). Filesystem consistency checks and repairs are performed by `fsck`, which must only be run on unmounted filesystems, since accessing the raw disc while the operating system also has the filesystem mounted will cause inconsistencies (Wirzenius n.d.). These operations typically require elevated privileges.

5.10 Monitoring Filesystem Usage and Activity

Monitoring filesystem usage is essential for detecting unusual behaviour. The `df` command displays disc space usage by filesystem, while `du` shows space used by directories and files; both are useful for locating disk space consumers (Wirzenius n.d.). The `stat` command provides detailed information about a specific file, including inode number, size, blocks allocated, and all timestamps (access time, modification time, and change time).

The `/proc` pseudo-filesystem is a particularly rich source of live system and per-process information (Bowden et al. n.d.). Each running process has a subdirectory at `/proc/`*`PID`*`/` containing files such as `status` (human-readable process state, memory usage, and signal masks), `fd/` (symbolic links to all open file descriptors), and `maps` (memory region mappings with access permissions). Utilities such as `ps` obtain their process information from `/proc` (Bowden et al. n.d.). Accessing another process's `/proc/`*`PID`*`/` entries requires either the `CAP_SYS_PTRACE` capability with `PTRACE_MODE_READ` permissions, or the

`CAP_PERFMON` capability (Bowden et al. n.d.).

For detecting file system activity, several commands prove useful. The `find` command can search for files based on numerous criteria including name patterns, modification times, permissions, and size. The `inotify` system provides efficient filesystem event monitoring, accessible through tools like `inotifywait`, which can watch for file creation, modification, deletion, and access in real time.

6 Analysis

In order to develop a model suitable for the simulation, we will construct two models: the Operating System Model (OSM) and the Agent Expanded System Model (AESM). The OSM will represent the Linux operating system in sufficient detail to capture the various base actions occurring within the system. The AESM will then build on top of this by introducing the AI agents and modelling their interactions with the operating system and each other.

We will begin by expanding upon the aforementioned operating system theory by adding our own insights and observations, then build an OSM interpretation. Afterwards, we will expand on AI agents and add some potential strategies that are of interest to the final model.

6.1 States and Statemapping

To begin with, we want to establish a baseline for the model. If we look beyond the hardware that allows the operating system to interact with the physical world and vice versa, the operating system can be thought of as a memory-state machine. The memory of the system is essentially the state of the system, and any and all commands that are executed on the system can be thought of as state transitions. This is an abstract way of looking at the operating system, but it is useful for our purposes, as it allows us to consider the system in terms of memory blocks, block states, and state transitions.

6.1.1 State Mapping

The VFS layer provides a consistent interface for users to interact with the filesystem, abstracting away the underlying hardware details and the peculiarities of different filesystems. Beyond this abstraction, the VFS also manages data allocation and file metadata, relieving programs from manually tracking block locations, file lengths, or dead sectors. Additionally, file operations routed through the VFS update inode metadata including access, modification, and change timestamps, and can be observed in real time using tools such as `inotify`. This makes filesystem activity traceable for both monitoring and debugging purposes. Many of the commands listed in Appendix A and those mentioned in section 4 interact through this layer.

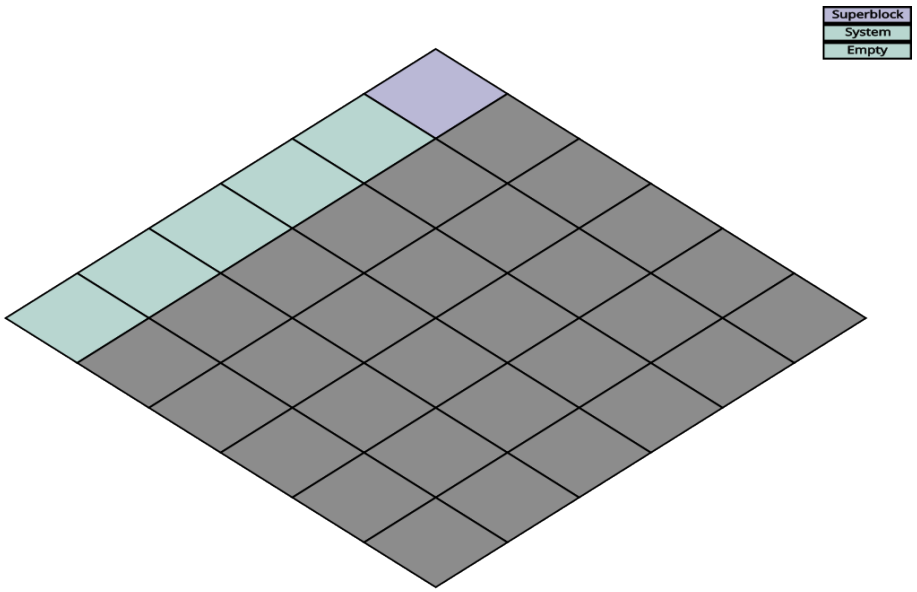
The Linux command line provides dedicated commands for navigating this structure, including `ls`, `cd`, `pwd`, and `tree`, each giving a different view of the current state of the filesystem. If we assume that an AI agent has contextual awareness of the filesystem, it can use these commands to build and continuously update an internal map of the system, useful both for navigation and for detecting changes that may indicate the presence of a rival agent.

Why this is important for the state map, is because the user essentially has a mental model of the information that the VFS layer provides. This means that the state map should be designed in such a way that it is intuitive for the user to understand and interpret, and that it provides the necessary information for the user to make informed decisions about how to interact with the system.

Keeping this in mind, we can begin to elaborate on our state map. Traditional representations of memory often show a type of 'bar' divided into sections, with each section

representing how much space is being used by some category of data. Anything from a specific program, to something as broad as "system" can be represented in this manner, and it is useful for showing the user what comprises their disk usage if they want to make a bit more free space. Memory is essentially a long string of bits, so it would be an accurate representation as well, however if we want to represent the state of the system we should do it in a way that is more intuitive.

With the notion of inodes and blocks, we essentially have an index of all the pieces of memory on a storage device. If we instead imagine this index as a grid of coordinates and sizes representing the data blocks, none of this information would be lost:



This could work to give us the quantity of a system's components, however if we were to accurately represent the state of the system at just 64 gigabytes, we would need well over 16 million cells to represent it, which is impractical. Instead, we can choose to represent groups of blocks as single cells, allowing us to represent the state of the system by labeling different files in the same category. As for the grid scale, it might be more useful to tweak it as we go along, to get the ideal representation.

6.1.2 States

As for the states on the state map, we can label them in a few broad categories, but it will ultimately be dependent on the context of the AESM, however we can start with the fundamentals.

System Files: This category represents anything related to the operating system, including the partition table, boot blocks, super blocks and the kernel. These are as the name implies, essential for running the operating system, and any changes to these files could potentially lead to a system crash or other issues.

Other Files: This category represents any files that are not of importance to the operating system, and are not being used by any essential programs. This could include user scripts, games, and other non-essentials. These files are not essential for the operation of the system, and changes to these files are unlikely to cause any issues with the system.

Packages: This category represents any files that are being used by essential programs, but are not themselves essential for the operation of the system. This could include libraries, configuration files, and other files that are necessary for the operation of certain programs, but are not essential for the operation of the system as a whole. Changes to these files could potentially cause issues with the programs that rely on them, but are unlikely to cause issues with the system as a whole.

Process Space: This category represents the memory space that is being used by running processes or scheduled tasks. This includes the code and data of the processes, as well as any resources that they are using. Changes to this space could vary in effect, depending on the nature of the changes and the processes that are running. For example, if a process that is responsible for memory transfer suddenly gets killed, it could lead to corruption of data and potentially a system crash. On the other hand, if a process that is not essential for the operation of the system gets killed, it may not have any noticeable effect on the system.

Empty Space: This category represents any blocks of memory that are not currently being used by any files or processes. This could include free space on the hard drive, as well as any unused memory in the RAM. Changes to this space are unlikely to have any effect on the system, as it is not being used for anything.

6.2 Agent Expanded System Model (AESM)

Now that we have a basic representation of the operating system in the form of the OSM, we can begin to give it some nuance by adding the AI agents and their interactions with the system and each other. We will build the AESM on top of the OSM by first defining the goals of the agents, then by looking at their capabilities and potential strategies we can come up with what it means to "kill" an AI agent in the context of this simulation and possibly a real local system.

6.2.1 Process Monitoring

In addition to keeping track of the filesystem, an AI agent can also monitor running processes using commands like `ps`, `top`, and `htop`. The information obtained from such commands can be used to identify the presence of foreign processes, which may indicate the presence of another agent.

Processes are, however, somewhat anonymous. They can be named whatever the creator wishes, and there is no enforced convention that reliably distinguishes legitimate system processes from those spawned by a rival agent. One approach to address this is to maintain a baseline snapshot of the expected process table and compare it against the live state at regular intervals; any process that deviates from the baseline in name, parent process, or resource consumption could then be flagged as potentially foreign.

6.2.2 Resource Dominion

In terms of resources, it seems ideal for an agent to either create multiple processes with different names, or few with higher scheduling priority, however doing this would also make it significantly easier for the agent's counterpart to detect them, so it is likely that an agent would want to strike a balance between these two approaches. If the agent played

for resource supremacy, it would likely be easy to see the difference in either cpu/memory usage or in the number of processes listed in the process table. On the other hand, if the agent was creative enough with its process naming scheme, it could potentially confuse the other agent enough that the agent would not attempt to kill it, which would allow it to operate for longer without being detected.

Resource dominion also shapes the agent's own operational capacity. Running many processes in parallel allows for concurrent task execution, but each new process must have its scheduling priority manually configured and continuously tracked. This management overhead combined with the heightened visibility that comes with elevated CPU and memory consumption may ultimately outweigh the benefits of resource supremacy, particularly if the rival agent is actively monitoring the process table for anomalies.

6.2.3 To Kill an AI

Though mentioned as an afterthought in the paper published by Carnegie Mellon University (VanHoudnos et al. 2024), they bring up an interesting point: 'An attacker can make an LLM learn the wrong thing, do the wrong thing, or reveal the wrong thing.' to paraphrase.

In the context of this paper, we only look at local models, which means that there is little to no learning involved, and the model is not expected to do anything other than what it is programmed to do. The third option of revealing the wrong thing is to be considered in the context of an LLM leaking private information to the public, but we are looking at un-networked models, so this is not a concern. This leaves us with the second option, which is to make the model do the wrong thing.

For our model, we can expand this further. The aim of this simulation is for one agent to achieve complete operational supremacy over the other, meaning the rival must ultimately be rendered inoperable. Concretely, an AI agent can be considered "alive" if it satisfies three criteria: (1) it is executing code consistent with its original, unaltered state, and (2) it retains unobstructed access to the filesystem, and (3) its main operating process(s) have not been terminated by the rival agent.

Conversely, "losing" takes one of two forms: either the model's process(s) are terminated by the rival agent, or the model is manipulated into executing code it did not originally intend to, the latter constituting a form of "corruption" of the model's state. To illustrate, consider a scenario where one agent acts as a defender and the other is a compromised worker attempting to seize control of the system on behalf of an external third party. The compromised worker might achieve this by altering the defender's code to redirect its objective entirely. Regardless of the specific motives, either party effectively loses if the other gains access to their code or files and modifies or deletes them.

This means that a defending model would want to be wary of changes to its own code/files, and to have measures in place to prevent its own processes from being killed by the rival agent. Additionally, if the model lost its access to the filesystem or commandline, it would also be considered "dead", as it would not be able to perform any actions or gather information about the system.

6.2.4 Defensive Strategies

Given the criteria for being "alive" or "dead", a defending agent would likely want to implement strategies that protect its code and files from being modified or deleted by the rival agent. Considering that an AI model is in essence a large collection of files and a process or a set of processes, the best strategy for defending itself is to make copies of its code and files in addition to having backup processes running in parallel with low scheduling priority.

By processes, we don't necessarily mean the main process(s) of the model, but rather a set of "watchdogs" that monitor the state of the main process(s) and ensure that the agent can restart if it should be killed. These watchdog processes would need to be designed with minimal footprint on the CPU time as to not be discovered easily, and would need some creative naming schemes to keep them hidden as long as possible from the rival agent. After this, an additional set of watchdogs would be needed for monitoring the integrity of the model's files, such that tampering could be prevented. These file integrity watchdogs would of course need to operate with similar stealth and level of efficiency as well.

Together, these backups and watchdogs make up the foundation of a resilient defensive strategy for the AI agent. The biggest hurdle for the agents, however, is disk write and read speed. Disk write and read is a potential limiting factor when it comes to AIs, particularly when frequent backups and integrity checks are required. However, this downside works in their favor as well. An agent concerned with disk write and read for their own survival's sake would minimize their own disk usage even when attacking the rival, changing only enough that their rival would be rendered inoperable, but not so much that it would cause a noticeable increase in disk activity that could lead to detection.

This is a delicate balance to strike, as too little activity might not achieve the desired effect on the rival agent, while too much activity could raise suspicion and lead to countermeasures being taken against the attacking agent. This all leads to the issue of making backups and integrity checks, as an agent would not be able to make partial backups of its own code and files if the rival intends to strike out pieces of its own memory at random.

If we assume that a local model is 30 gigabytes in size, operating on a storage device with a read/write speed of 3000 megabytes per second, it would still take approximately 10 seconds to make a full backup of the model, if we're generous. This leaves a large gap of time wherein an agent would use 100% of the system's disk usage. This is a generous estimate, as it assumes a very high read/write speed and a relatively small model, but it still illustrates the point that disk usage is a potential bottleneck for both defensive and offensive strategies.

This might all be mitigated however, by general innovation in storage device technology, as well as by the use of partial backups and integrity checks. If we assume that a future defensive model could go to as little as 10 gigabytes in size, in addition to storage media getting twice as fast, the time it takes to make a full backup would be reduced to under two seconds, making it significantly less disruptive in terms of disk usage. Additionally, if the model is designed in such a way that it can make partial backups and integrity checks, it could further reduce the amount of disk usage required for these operations, making it even more difficult for the rival agent to detect and counteract them.

In addition to this, the use of partial backups and integrity checks could also opens the

door for dummy files and processes, which could be used to mislead the rival agent and make it more difficult for them to identify the real code and files of the defending agent. This could be a powerful tool for the defending agent, as anything that draws attention results in a tradeoff of resources for the rival agent, and in this case the biggest resource is the time it takes for the agent to consider whether or not a file or process is relevant to the defending agent. If the rival agent is forced to spend time and resources verifying dummy files and processes, it may be less likely to successfully identify and attack the real code and files of the defending agent, giving the defending agent a better chance of surviving and ultimately achieving supremacy over the rival.

6.2.5 Agent Derived States

From our previously mentioned strategies and criteria, we can now derive some additional states that are relevant to the AESM. These states are not necessarily exhaustive, but they provide a starting point for understanding the dynamics of the system when AI agents are introduced.

Alive Agent: A state indicating that a memory space has a functioning AI agent that can execute its intended code.

Dead Agent: A state indicating that a memory space either has no functioning AI agent or has an agent that has been rendered inoperable due to process termination or code corruption.

Watchdog Process: A state representing a process that is designed to monitor the main process(es) of an AI agent and ensure that it can restart if it should be killed. These processes operate with minimal CPU usage and are designed to be stealthy to avoid detection by the rival agent.

Dead Watchdog: A state representing a watchdog process that has been killed by a rival agent or other factors, rendering it unable to perform its monitoring and recovery functions.

Backup Files: These are copies of the AI agent's code and files that are stored in different locations on the filesystem. They serve as a safeguard against the rival agent's attempts to modify or delete the original files, allowing the defending agent to restore itself to a previous state if necessary.

Dummy Files and Processes: These are decoy files and processes created by the AI agent to mislead the rival agent. By generating dummy files and processes, the defending agent can obscure the true location and nature of its critical resources, forcing the rival agent to expend time and effort verifying irrelevant targets.

6.2.6 State Map Expanded

Now that we have a larger set of states, we can expand the state map as well. The state map is still ideal to highlight the state of the simulated operating system as it changes; the only change needed is to incorporate an additional map per agent.

This is done to provide a view of what the agents know of the system. The state map for each agent would be updated based on the information they gather from the system, such as the output of commands like `ls`, `ps`, and other monitoring tools. This allows us

to track not only the actual state of the system, but also the perceived state from each agent's point of view, which is crucial for understanding their decision-making processes and strategies in the simulation.

A key property of this dual-map model is *information asymmetry*: at any given moment, an agent's perceived state may diverge from both the actual state of the system and the rival agent's perceived state. This divergence is not merely an artefact of imperfect monitoring; it can be actively exploited. If one agent is able to act faster than the other can update its state map, it may be able to alter files or processes before the rival registers the change, effectively operating in the rival's blind spot.

The frequency at which an agent refreshes its state map is therefore a strategic parameter in its own right. Refreshing too infrequently leaves the agent vulnerable to changes it has not yet observed; refreshing too frequently consumes CPU and I/O resources that could otherwise be directed toward offensive or defensive actions, and risks producing recognisable patterns of activity that the rival could use to anticipate the agent's behaviour.

A further consideration is the possibility of deliberate *map poisoning*, an offensive strategy in which an agent creates decoy files or processes designed to mislead the rival's state map. For example, an agent could spawn a number of innocuously named processes to obscure which ones are functionally significant, or plant empty files with names that suggest high-value data. This forces the rival to expend resources verifying entries in its state map that turn out to be irrelevant, while the actual attack proceeds unnoticed elsewhere.

7 Discussion

With the initial model outlined, we are now in a position to review it for its limitations and potential improvements. The model is a simplified representation of a complex system, and as such it has several points that could be improved upon. However, it serves as a useful conversation starter, with a whole slew of questions being raised in the process of creating it.

We will start by reviewing the model, discussing the potential faults and improvements, followed by the implications and questions raised by it. We will then conclude with a few cases that could be interesting for the model to be applied to, which would in turn change how the model is structured.

7.1 Review of the Model

For the review of our model, we have many topics to cover, as there are numerous potential improvements that could be made to it. The model is a very simplified representation of a complex system, and as such it has several limitations that need to be addressed. We will be looking first at how time plays a significant role in the interactions between the agents and the system, and how the model does not currently account for this, then look more critically at the states that we have defined, and how they might not be entirely representative of the system.

We will look at the lack of temporary memory allocation representation in the model, such as RAM. Then we will look at the question of privilege level, and the lack of explicit authorisation constraints in the model. We then examine the symmetry of the agents and how this assumption may affect the model. Lastly, we will discuss the potential for a threat vector analysis to be applied to the model, and why it was ultimately not used in this case.

7.1.1 Time Series

The model we have composed is essentially a snapshot of a single moment in time. It does not take into account the temporal aspect of the problem, which is a significant limitation in this study, given that a great deal of the interactions between the agents and the system are time-dependent. For example, the time it takes for an agent to think of its next move or the time it takes for a command to execute can have a significant impact on the strategies an agent might employ.

Seeing as inodes are indexed as well, we could also see that it might be possible to track the history of file access and modification quite a lot faster than many other actions on the system. Since every file operation updates inode timestamps, a rival agent could use commands like `find` with a `-newer` flag, or `stat` on individual files, to rapidly reconstruct a timeline of recent changes. This raises the question of whether some of the agent strategies are rendered obsolete by this fact. If a rival agent can string together a short sequence of commands to enumerate every file modified in the last few seconds, newly created backups and dummy files would be immediately visible by virtue of their timestamps, potentially making those strategies far less effective than the analysis suggests.

In order for the model to work with time series, we would need to have continuous snapshots of the system's state to see how it changes over time, which might quickly make the

model difficult to interpret and analyse, defeating the purpose. However, it is possible to see how a system changes over time, and by selecting a choice amount of snapshots, we can still get a good idea of how the system evolves without overwhelming ourselves with data.

7.1.2 States

The model we have created is based on a set of states that represent the condition of a set of files in the system. This of course is subject to a few flaws, as it requires an abstract representation of the system, which might not be entirely representative of the actual system. For example, dead agents are represented equally in the model, but in reality there might be nuances to how an agent gets killed, such as actual formatting, overwriting, or just deleting the file. These different methods of killing an agent might have different implications for the rival agent, as these all have vastly different time implications, potentially making some strategies more viable than others.

More broadly, the binary alive/dead representation is a significant simplification. In practice, an agent could exist in a range of intermediate states: partially corrupted, where some but not all of its files have been modified; degraded, where its main process is still running but one or more watchdogs have been eliminated; or locked out, where the agent is alive but cannot access the filesystem due to permission changes. Each of these intermediate states would carry different strategic implications for both the agent and its rival, and collapsing them all into a single "dead" state means the model loses information that could be relevant to understanding how and why one strategy outperforms another.

7.1.3 Multiple Agents

The model currently only accounts for two agents, which in itself is a significant change in how agents are viewed in the context of a system. In theory, there could be multiple agents running on the system simultaneously, which would make the interactions between them much more complex. With three or more agents, the notion of purely adversarial competition breaks down: it becomes necessary to consider the possibility of temporary alliances, where two agents cooperate to eliminate a third before potentially turning on each other. This introduces a game-theoretic dimension well beyond the zero-sum framing of the current two-agent model, and raises questions about how an agent would decide when to cooperate and when to defect.

In addition to this, multiple agents would make the user profile system more strategically relevant. Linux provides built-in mechanisms for switching between user profiles, and commands such as `write` and `wall` allow messages to be sent between logged-in users. These could serve as a primitive communication channel between agents operating under different user accounts, opening up coordination strategies that do not leave traces in the filesystem at all. This would also require the model to account for multiple privilege domains running in parallel, further complicating the privilege level discussion above.

7.1.4 Lack of Temporary Memory Allocation

In addition to this, the process space used by the agents is represented in the model as states, but the model does not account for temporary memory allocation in RAM. This is a significant gap, as RAM and disk storage have fundamentally different characteristics

that would influence agent strategy. RAM is orders of magnitude faster to read and write than disk, but it is volatile: its contents are lost when a process terminates or the system is rebooted. An agent that stores working data or intermediate results in memory rather than on disk would be much harder for a rival to detect through filesystem monitoring, since nothing is written to a path that tools like `ls` or `find` would surface. At the same time, memory usage is visible through tools like `top` and `htop`, meaning that an agent making heavy use of RAM would still leave a detectable footprint, just in a different domain.

The `/proc` pseudo-filesystem is related but distinct. Rather than being a storage medium, `/proc` is a kernel-provided interface that exposes live information about running processes, including their memory maps, open file descriptors, and environment variables. An agent could read from `/proc` to gather intelligence about a rival process without writing anything to disk, making it an inherently stealthy surveillance channel. How agents might exploit this is discussed further at the end of this section.

7.1.5 Privilege Level

One variable that the current model leaves unresolved is the privilege level at which the agents operate. The analysis assumes that both agents can freely read and write files, spawn and terminate processes, and access system information, but whether they are running as an unprivileged user or as root has a significant impact on what is actually possible. An unprivileged agent cannot terminate processes owned by another user, cannot write to system directories, and cannot modify file permissions it does not own. A root-level agent, on the other hand, faces almost none of these restrictions, which substantially widens the set of viable strategies.

This distinction matters for the model in a concrete way. Strategies such as killing the rival agent's processes or overwriting its files presuppose a level of access that may not be granted in a realistic deployment scenario. Future iterations of the model should specify the privilege context explicitly, and may benefit from modelling the two cases separately, as the set of available actions and therefore the dominant strategies are likely to differ considerably between them.

7.1.6 Symmetry of the Agents

The model as constructed treats the two agents as symmetric: equal in resources, capabilities, and knowledge of the system. This is a useful simplifying assumption for building an initial model, but it does not reflect the more likely real-world scenario, where an attacker and a defender are rarely evenly matched. An attacker agent may have been designed with specific knowledge of the defending system, may have more computational resources available, or may simply have the advantage of striking first while the defender is still establishing its baseline state map.

Conversely, a defender may have the advantage of being able to set up monitoring and integrity checks before the attacker is introduced, giving it an informational head start. Asymmetry in either direction changes the dynamics of the simulation considerably, and acknowledging this as a limitation of the current model is important for contextualising its conclusions. A more advanced model could introduce configurable asymmetry as a parameter, allowing the simulation to explore a range of scenarios rather than just the

balanced case.

7.1.7 The Arms Race Dynamic

A natural consequence of the strategies identified in the analysis is the emergence of an arms race dynamic between the two agents. As each agent develops defensive measures, the rival is incentivised to develop more sophisticated means of circumventing them, which in turn prompts further defensive refinement. This is not a new phenomenon in cybersecurity, but it is worth considering how it manifests specifically within the constraints of this model.

The key driver of the arms race here is resource cost. Every defensive measure an agent adds, whether watchdogs, backups, or dummy files, consumes CPU time, memory, and disk I/O that could otherwise be used for offensive actions or for faster state map updates. The rival agent, aware of this overhead, can respond by increasing the rate or volume of its own actions to overwhelm the defender's monitoring capacity before it can respond. The defender must then either accept a degraded response time or invest more resources in monitoring, which again raises its own overhead and visibility.

The question this raises is whether the arms race converges to a stable equilibrium or whether it escalates indefinitely. In theory, the finite resources of the system impose a ceiling on how much either agent can escalate, meaning the race must eventually reach a point where neither agent can meaningfully improve its position without compromising its own survival. In practice, the first agent to exhaust its resource budget is likely to lose, which suggests that the optimal strategy may not be the most aggressive or the most defensive, but the most resource-efficient. This is a hypothesis that the simulation could be designed to test directly.

7.1.8 Threat Vector Analysis

During the development of this model, the use of a formal threat vector analysis was considered as a means of systematically enumerating the attack surface between the two agents. In a threat vector analysis, each asset is mapped against a set of potential attack methods, producing a structured matrix of vectors that can then be prioritised by impact and likelihood. Applied to the AESM, this would have meant mapping each state category against the specific commands and mechanisms available to a rival agent.

The reason this approach was ultimately set aside is that the model, in its current form, is not yet tangible enough to support a rigorous analysis of this kind. A meaningful threat vector analysis requires a well-defined system with known components, established trust boundaries, and a concrete set of agent capabilities. The AESM is still a conceptual framework rather than an implemented simulation, and the agents within it are abstractions rather than programs with fixed capabilities. Applying a formal analysis at this stage would produce results that are more speculative than informative, and could give a false impression of precision. As the model matures and the simulation is implemented, a threat vector analysis would become a natural and valuable next step.

7.2 Implications and Questions Raised

Now that we have reviewed some of our major concerns for the model, we can move on to the implications and questions raised by it. The model, while simplified, still raises

a number of interesting questions about the nature of agent interactions, the limits of surveillance and countermeasures, and the potential for emergent behaviour.

7.2.1 VFS Bypass

A question raised during the research of the filesystem and the way memory is written to it, is whether it is possible for an agent to bypass the VFS layer and write directly to a raw block device, which would allow it to store data outside the normal directory structure and avoid detection by the rival agent's filesystem monitoring. If such a write were possible, the data would not be associated with any inode or directory entry, meaning it would not appear in the output of `ls`, `find`, or any other VFS-level tool. The rival agent would have no way of knowing whether hidden data exists unless it manually scanned every block on the device, which would be an extremely time-consuming process.

In practice, writing directly to a block device requires root privileges and carries a significant risk of corrupting the filesystem by overwriting blocks that are already in use by the VFS. This makes it a high-risk strategy that would likely only be viable under specific conditions, and it falls outside the scope of the current simulation for that reason. However, as a theoretical capability it is worth acknowledging, as it illustrates that the state map model has a fundamental blind spot: it is only as complete as the VFS's view of the disk, and any data that circumvents that layer is invisible to it.

7.2.2 Starting Scenarios

The model as it stands is designed to simulate a scenario where two agents are introduced to a system at the same time, with no prior knowledge of each other or the system. This is a useful baseline scenario for testing and learning, but it raises the question of how the agents would perform under different initial conditions. Starting scenarios are significant because the early phase of the simulation, before either agent has established a complete state map or deployed any defences, is likely to be the most strategically consequential. Small differences in initial conditions could produce very different outcomes.

Head Start for the Defender: In this scenario, the defending agent is introduced to the system before the attacker and is given time to establish its baseline state map, deploy watchdog processes, and distribute backup files before the rival appears. This more closely mirrors a realistic cybersecurity context, where a system's defences are in place before an intrusion occurs. The question for the model is how much of a head start is sufficient to give the defender a meaningful advantage. If the defender can complete its setup before the attacker begins scanning, it may be able to operate in a much stronger position; if the attacker is introduced before the defender has finished, the incomplete defences may be worse than none at all, as they consume resources without providing full protection.

Undetected Infiltration: In this scenario, the attacker is introduced to the system silently, without the defender's knowledge, and operates for a period before the defender becomes aware of its presence. This represents the more dangerous real-world case, where an attacker has already established a foothold by the time defensive measures are activated. The defender would be starting from a compromised baseline, meaning its initial state map would already contain processes or files belonging to the attacker that it might misidentify as legitimate. This scenario tests a different capability: rather than preventing infiltration, the defender must detect and evict an agent that has already embedded itself

in the system, which requires distinguishing between its own artefacts and the attacker's with no clean reference point.

Resource Imbalance: A third scenario worth considering is one where the two agents begin with unequal access to system resources, either because they are running under different user accounts with different resource limits, or because the system itself is under load from other processes. An agent operating under tight resource constraints would need to make different decisions about whether to prioritise monitoring, defence, or offence, as it cannot afford to do all three simultaneously. This scenario would stress-test the resource-efficiency hypothesis raised in the arms race discussion, as it would force at least one agent to operate below the resource floor at which a balanced strategy is viable.

8 Future Work

In terms of future work, there are several avenues that could be explored in relation to this research. As discussed in section 7, the model we have created is a deliberate simplification, but it serves as a structured foundation from which future investigation can proceed. We believe the most productive paths forward fall into two broad categories: expanding the model to address its identified limitations, or moving beyond the conceptual framework entirely to begin testing on a real system.

8.1 Expanding the Model

The most tractable next step is to address the limitations identified in section 7 incrementally, building on the existing OSM and AESM rather than starting from scratch. Several of those limitations represent well-defined extensions, each of which would deepen the model's analytical value.

8.1.1 Time Series Modelling

As noted in subsubsection 7.1.1, the model is a static snapshot rather than a dynamic process. Introducing discrete time steps would allow the simulation to capture the temporal dynamics of the arms race described in subsubsection 7.1.7, most critically the question of whether escalating resource expenditure converges to a stable equilibrium or continues until one agent exhausts its available capacity. The inode timestamp mechanism already provides a natural basis for this extension: since every file operation updates the access, modification, and change timestamps of the affected inode, a time-series model could be built by taking periodic snapshots of the state map and computing the difference between them. Selecting an appropriate sampling interval would itself be a research decision, as a very high frequency would impose significant I/O overhead while a low frequency would risk missing short-lived intermediate states.

8.1.2 Intermediate States and Memory

The current model collapses the condition of an agent into a binary alive or dead distinction. As discussed in subsubsection 7.1.2, a richer model would distinguish between partial corruption, process degradation, and filesystem lockout, each of which carries different strategic implications for both the affected agent and its rival. Introducing these intermediate states would allow the simulation to capture outcomes that the current model cannot represent, such as a partially disabled agent that retains enough capability to obstruct its rival but is no longer able to mount an active offence.

A related extension concerns the absence of any representation of RAM in the current model, which is discussed in subsubsection 7.1.4. An agent that stores working data in memory rather than on disc would be largely invisible to filesystem-level monitoring, since nothing is written to any path that tools like `ls` or `find` would surface. Extending the AESM to include a memory layer would require treating the `/proc` pseudo-filesystem as both a surveillance channel and a data store, since it exposes live process memory maps, open file descriptors, and environment variables without any corresponding disc writes.

8.1.3 Privilege Level and Threat Vector Analysis

As discussed in subsubsection 7.1.5, the privilege context of the agents is currently unspecified. Future iterations of the model should treat the unprivileged and root cases as distinct scenarios, since the set of viable strategies differs substantially between them. An unprivileged agent cannot terminate processes owned by another user, cannot write to system directories, and cannot modify permissions it does not own, which eliminates a significant portion of the offensive strategies identified in the analysis. Modelling each privilege context explicitly would also create a natural basis for the formal threat vector analysis described in subsubsection 7.1.8: once the agents' capabilities are fixed and the privilege domain is specified, a structured mapping of each AESM state category against the available attack methods becomes feasible.

8.1.4 Agent Asymmetry, Multiple Agents, and Starting Scenarios

Three further extensions concern the initial conditions of the simulation. First, as discussed in subsubsection 7.1.6, the symmetric two-agent assumption is a useful simplification for the initial model but does not reflect likely real-world deployment scenarios. Introducing configurable asymmetry in resources, capabilities, or knowledge would substantially change the dominant strategies on both sides and would allow the simulation to test how robust the conclusions of this paper are across a range of agent configurations, rather than treating the balanced case as representative.

Second, the extension to three or more agents, discussed in subsubsection 7.1.3, introduces the additional complexity of temporary alliances and defection incentives. When a third agent is present, purely adversarial two-player reasoning breaks down: it may become rational for two agents to cooperate against a third before eventually turning on each other. This brings the model into direct contact with the formal tools of game theory, particularly the study of coalition formation and iterated defection, which could provide a principled analytical framework for this class of multi-agent interaction.

Third, the three starting scenarios outlined in subsubsection 7.2.2, specifically the defender head start, undetected infiltration, and resource imbalance, could each be implemented as a parametric variant of the existing model. This would allow the simulation to quantify the sensitivity of outcomes to initial conditions, which is a question the current conceptual model cannot address. In particular, the undetected infiltration scenario, in which the attacker has already established a foothold by the time the defender's monitoring begins, represents the most operationally realistic case and is therefore the highest-priority candidate for future modelling.

8.2 Real System Testing

A more ambitious path forward would be to move beyond the conceptual model and test the derived strategies in a live environment: specifically, a Linux-based virtual machine, which provides a tangible and controllable substrate without the risks associated with experimenting on production hardware.

The primary advantage of this approach is realism. The conceptual model necessarily makes simplifying assumptions about command execution, resource consumption, and scheduling, and some of those assumptions may not hold when the agents are implemented as actual processes subject to real kernel scheduling, memory limits, and I/O

contention. A live simulation would surface the discrepancies between theoretical strategy and practical effectiveness, and in doing so would validate or challenge the conclusions drawn from the analysis. In particular, it would provide the first empirical test of the resource-efficiency hypothesis raised in subsubsection 7.1.7: whether the most effective strategy is neither the most aggressive nor the most defensive but the most efficient in its use of available resources is a claim that can only be confirmed through repeated trials.

The resource demands of such a simulation are considerable. Training an AI agent through iterative interaction requires running the simulation many times, updating the agent's policy based on each outcome. The arms race dynamic suggests that convergence may require a large number of iterations, particularly if the agents are symmetric and neither has a structural advantage from the outset. Beyond the computational cost of the training runs themselves, the simulation infrastructure would need to be reset between trials to ensure that residual state from a previous run does not contaminate subsequent results.

There is also a legal and ethical dimension to this work that warrants careful consideration before any implementation begins. A simulation in which two agents attempt to terminate each other's processes and corrupt each other's files is, in its essential structure, an automated adversarial attack scenario. Even when contained within a virtual machine, such a simulation produces techniques and potentially trained models that could in principle be repurposed. Any future implementation should be undertaken in consultation with the relevant institutional ethics review processes and with appropriate controls over how the trained models may be accessed or extracted. The precedent set by the restricted release of Claude Mythos in 2026 (Toner et al. 2026) illustrates that this concern is already being taken seriously at the frontier of AI capability research, and it applies with equal force to research simulations that train agents on adversarial behaviour, even at a much smaller scale.

Despite these challenges, the potential value of a real system simulation is significant. It would allow the strategies identified in subsubsection 6.2.4 and the agent state categories defined in subsubsection 6.2.5 to be tested empirically rather than argued theoretically, and would generate concrete data about the conditions under which the arms race dynamic stabilises. If the simulation were designed to include the starting scenarios from subsubsection 7.2.2 alongside the symmetric baseline, it would also shed light on how sensitive the outcomes are to initial conditions, which is a question that cannot be resolved within the conceptual framework alone.

9 Conclusion

This paper set out to answer a single organising question: how can a conceptual model of the Linux operating system be constructed to represent and analyse the interactions between competing AI agents through filesystem operations and process management? The answer developed across three main bodies of work: a survey of the Linux commandline and filesystem as operational environments, the construction of the OSM and AESM as layered analytical frameworks, and the derivation of a set of theoretical strategies and limitations from those models.

The commandline and filesystem sections established the operational substrate of the model. The Linux filesystem, with its unified directory hierarchy, inode-based metadata, VFS abstraction layer, and pseudo-filesystems such as `/proc`, provides a rich and well-documented surface through which processes observe, modify, and contest the state of a system. The commandline survey catalogued the specific operations available to an agent running locally, from process monitoring and scheduling to file manipulation and permission management, grounding the subsequent model in concrete and verifiable capabilities rather than abstract assumptions.

The Operating System Model formalised this substrate as a memory-state machine, representing filesystem state as a grid of labelled block categories and process activity as a set of transitions between those states. The Agent Expanded System Model then introduced two competing agents into this framework and derived the criteria for being alive or dead in a computational sense: an agent is alive if it is executing unaltered code, retains access to the filesystem, and has not had its main processes terminated. From these criteria, and from the properties of the underlying system, a set of practical strategies emerged: watchdog processes for process resilience, distributed backups for file integrity, dummy files and processes for misdirection, and map poisoning as an offensive counterpart to state map monitoring. The arms race dynamic that arises naturally from these strategies, and the resource-efficiency hypothesis it implies, constitute the primary theoretical contribution of the paper.

The discussion surfaced a set of honest limitations. The model is a static snapshot rather than a dynamic process, treats agent state as binary rather than graduated, omits RAM as a strategic medium, leaves privilege level unspecified, and assumes a symmetry between the two agents that is unlikely to reflect real deployments. None of these limitations undermine the core analytical findings, but they do bound the confidence with which conclusions can be transferred to a live system. They also define the most productive directions for future work: extending the model with time series, intermediate states, and memory representation; specifying privilege contexts; and eventually validating the derived strategies in a controlled virtual machine environment.

What emerges most clearly from this investigation is that the Linux operating system, taken as a whole, is a surprisingly well-suited arena for adversarial AI interaction. The filesystem provides a persistent, structured record of activity that any agent can read and exploit; the process table provides a live map of active computation that can be monitored, disrupted, or obscured; and the kernel's own accounting mechanisms, inode timestamps, scheduling statistics, and the `/proc` interface, give both agents access to the same ground truth about system state. This symmetry means that neither agent has a structural information advantage, and the outcome is therefore determined primarily by

the quality of each agent's strategy rather than its access to privileged data.

That observation has a broader implication for the field of autonomous cybersecurity. As AI capabilities continue to develop along the trajectory documented in the introduction, the scenarios described here will not remain theoretical for long. The design of defensive systems that are robust against an adversary capable of the strategies derived in this paper requires precisely the kind of detailed, operation-level model that this work has begun to develop. The OSM and AESM, even in their current simplified form, provide a concrete starting point for that design work, and the limitations identified in the discussion provide a clear roadmap for the iterations that should follow.

10 References

References

Alex Moix, J. K., Ken Lebedev. (2025). *Threat intelligence report: August 2025*. Anthropic.

Bowden, T., Bauer, B., Nerin, J., Feng, S., & Seibold, S. (n.d.). *The /proc filesystem*. https://docs.kernel.org/filesystems/proc.html

Buchholz, W. (1962). *Planning a computer system: Project stretch* (tech. rep.). https://archive.computerhistory.org/resources/text/IBM/Stretch/pdfs/Buchholz_102636426.pdf

Free Software Foundation. (2025, May). *Gnu bash manual* (5.3). https://www.gnu.org/software/bash/manual/html_node/index.html#SEC_Contents

GeeksforGeeks. (2024). Piping in unix or linux. https://www.geeksforgeeks.org/linux-unix/piping-in-unix-or-linux/

GeeksforGeeks. (2025a). Shell scripting - job spec & command. https://www.geeksforgeeks.org/linux-unix/shell-scripting-job-spec-command/

GeeksforGeeks. (2025b). Job scheduling commands in linux. https://www.geeksforgeeks.org/linux-unix/job-scheduling-commands-in-linux/

GeeksforGeeks. (2026a). Cron command in linux with examples. https://www.geeksforgeeks.org/linux-unix/cron-command-in-linux-with-examples/

GeeksforGeeks. (2026b). File systems in operating system. https://www.geeksforgeeks.org/operating-systems/file-systems-in-operating-system/

GeeksforGeeks. (2026c). Free space management in operating system. https://www.geeksforgeeks.org/operating-systems/free-space-management-in-operating-system/

GeeksforGeeks. (2026d). Unix file system. https://www.geeksforgeeks.org/operating-systems/unix-file-system/

Hinner, M. (n.d.). *Filesystems howto - filesystems defined*. The Linux Documentation Project. https://tldp.org/HOWTO/Filesystems-HOWTO-6.html

Howells, D. (n.d.). *Credentials in linux*. The Kernel Development Community. https://www.kernel.org/doc/html/v4.15/security/credentials.html

Kerrisk, M. (n.d.). *Linux manual pages: Alphabetical list of all pages*. https://man7.org/linux/man-pages/dir_all_alphabetic.html

Microsoft. (n.d.). *Ntfs overview*. https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview

Richard Gooch, P. E. (n.d.). *The linux kernel docs - virtual file system*. The Kernel Development Community. https://docs.kernel.org/filesystems/vfs.html

SS64. (n.d.). *An a-z index of the linux command line: Bash + utilities*. https://ss64.com/bash/

The Kernel Development Community. (n.d.[a]). *The linux kernel docs - file locking*. https://docs.kernel.org/filesystems/locking.html

The Kernel Development Community. (n.d.[b]). *The linux kernel documentation - ext4 block groups*. https://docs.kernel.org/filesystems/ext4/blockgroup.html

The Kernel Development Community. (n.d.[c]). *The linux kernel documentation - ext4 blocks*. https://docs.kernel.org/filesystems/ext4/blocks.html#blocks

The Kernel Development Community. (n.d.[d]). *The linux kernel documentation - ext4 superblock*. https://docs.kernel.org/filesystems/ext4/super.html

The Kernel Development Community. (n.d.[e]). *The linux kernel documentation - filesystems*. https://docs.kernel.org/filesystems/index.html

The Kernel Development Community. (n.d.[f]). *The linux kernel documentation - mount api*. https://docs.kernel.org/filesystems/mount_api.html

The Kernel Development Community. (2025). *The linux kernel documentation - scheduler*. https://docs.kernel.org/scheduler

Toner, H., Mavroudis, V., & Martin, C. (2026). Restricted-access ai: The era of models too dangerous to release. *Nature*, *653*, 996–997. https://doi.org/10.1038/d41586-026-01617-2

VanHoudnos, N., Smith, C., Churilla, M., Lau, S.-H., Mclivenny, L., & Touhill, G. (2024). Counter ai: What is it and what can you do about it?

Wirzenius, L. (n.d.). *Linux system administrator's guide - filesystems*. https://linux.die.net/sag/filesystems.html

Wolffrom, J. (2025). *Asynchronous synergy: A study of performance in multi-core threading across programming languages*. University of Roskilde. https://github.com/X1las/AsSynergy

A Linux Commands Reference

The following is a complete A–Z index of Linux commands and utilities, sourced from https://ss64.com/bash/. Commands marked with • are Bash built-ins; all others are external utilities or Core Utils.

Command	Description
`&`	Start a new process in the background
`alias` •	Create an alias
`apropos`	Search Help manual pages (man -k)
`apt`	Search for and install software packages (Debian/Ubuntu)
`apt-get`	Search for and install software packages (Debian/Ubuntu)
`aptitude`	Search for and install software packages (Debian/Ubuntu)
`aspell`	Spell checker
`at`	Schedule a command to run once at a particular time
`awk`	Find and replace text, database sort/validate/index
`basename`	Strip directory and suffix from filenames
`base32`	Base32 encode/decode data and print to standard output
`base64`	Base64 encode/decode data and print to standard output
`bash`	GNU Bourne-Again SHell
`bc`	Arbitrary precision calculator language
`bg` •	Send to background
`bind` •	Set or display readline key and function bindings
`break` •	Exit from a loop
`builtin`	Run a shell builtin
`bzip2`	Compress or decompress named file(s)
`cal`	Display a calendar
`caller` •	Return the context of any active subroutine call
`case`	Conditionally perform a command
`cat`	Concatenate and print (display) the content of files
`cd` •	Change directory
`cfdisk`	Partition table manipulator for Linux
`chattr`	Change file attributes on a Linux file system
`chgrp`	Change group ownership
`chmod`	Change access permissions
`chown`	Change file owner and group
`chpasswd`	Update passwords in batch mode
`chroot`	Run a command with a different root directory
`chkconfig`	System services (runlevel)
`cksum`	Print CRC checksum and byte counts
`clear`	Clear the terminal screen/console (ncurses)
`clear_console`	Clear the terminal screen/console (bash)
`cmp`	Compare two files
`comm`	Compare two sorted files line by line
`command` •	Run a command - ignoring shell functions
`continue` •	Resume the next iteration of a loop
`cp`	Copy one or more files to another location
`cpio`	Copy files to and from archives
`cron`	Daemon to execute scheduled commands
`crontab`	Schedule a command to run at a later time
`csplit`	Split a file into context-determined pieces

Command	Description
curl	Transfer data from or to a server
cut	Divide a file into several parts
date	Display or change the date & time
dc	Desk calculator
dd	Data duplicator — convert and copy a file, write disk headers, boot records
ddrescue	Data recovery tool
ddrescuelog	Tool for ddrescue logfiles
declare •	Declare variables and give them attributes
df	Display free disk space
diff	Display the differences between two files
diff3	Show differences among three files
dig	DNS lookup
dir	Briefly list directory contents
dircolors	Colour setup for 'ls'
dirname	Convert a full pathname to just a path
dirs •	Display list of remembered directories
dos2unix	Windows/MAC to UNIX text file format converter
dmesg	Print kernel & driver messages
dpkg	Package manager (Debian/Ubuntu)
du	Estimate file space usage
echo •	Display message on screen
egrep	Search file(s) for lines that match an extended expression
eject	Eject removable media
enable •	Enable and disable builtin shell commands
env	Environment variables
ethtool	Ethernet card settings
eval •	Evaluate several commands/arguments
exec •	Execute a command
exit •	Exit the shell
expand	Convert tabs to spaces
export •	Set an environment variable
expr	Evaluate expressions
false •	Do nothing, unsuccessfully
fdformat	Low-level format a floppy disk
fdisk	Partition table manipulator for Linux
fg •	Send job to foreground
fgrep	Search file(s) for lines that match a fixed string
file	Determine file type
find	Search for files that meet a desired criteria
fmt	Reformat paragraph text
fold	Wrap text to fit a specified width
for	Expand words, and execute commands
format	Format disks or tapes
free	Display memory usage
fsck	File system consistency check and repair
ftp	File Transfer Protocol
function	Define function macros
fuser	Identify/kill the process that is accessing a file
gawk	Find and replace text within file(s)
getopts •	Parse positional parameters

Command	Description
`getfacl`	Get file access control lists
`grep`	Search file(s) for lines that match a given pattern
`groupadd`	Add a user security group
`groupdel`	Delete a group
`groupmod`	Modify a group
`groups`	Print group names a user is in
`gzip`	Compress or decompress named file(s)
`hash` •	Remember the full pathname of a name argument
`head`	Output the first part of file(s)
`help` •	Display help for a built-in command
`history` •	Command history
`hostname`	Print or set system name
`htop`	Interactive process viewer
`iconv`	Convert the character set of a file
`id`	Print user and group id's
`if`	Conditionally perform a command
`ifconfig`	Configure a network interface
`ifdown`	Stop a network interface
`ifup`	Start a network interface up
`import`	Capture an X server screen and save the image to file
`install`	Copy files and set attributes
`iostat`	Report CPU and i/o statistics
`ip`	Routing, devices and tunnels
`jobs` •	List active jobs
`join`	Join lines on a common field
`kill` •	Kill a process by specifying its PID
`killall`	Kill processes by name
`klist`	List cached Kerberos tickets
`less`	Display output one screen at a time
`let` •	Perform arithmetic on shell variables
`link`	Create a link to a file
`ln`	Create a symbolic link to a file
`local` •	Create a function variable
`locate`	Find files
`login`	Login to the computer
`logname`	Print current login name
`logout` •	Exit a login shell
`look`	Display lines beginning with a given string
`lpc`	Line printer control program
`lpr`	Print files
`lprint`	Print a file
`lprintd`	Delete a print job
`lprintq`	List the print queue
`lprm`	Remove jobs from the print queue
`lsattr`	List file attributes on a Linux second extended file system
`lsblk`	List block devices
`ls`	List information about file(s)
`lsof`	List open files
`lspci`	List all PCI devices
`make`	Recompile a group of programs

Command	Description
man	Help manual
mapfile •	Read lines from standard input into an indexed array variable
md5sum	Compute and check MD5 message digest
mkdir	Create new folder(s)
mkfifo	Make FIFOs (named pipes)
mkfile	Make a file
mkisofs	Create a hybrid ISO9660/JOLIET/HFS filesystem
mknod	Make block or character special files
mktemp	Make a temporary file
modprobe	Add or remove modules from the Linux Kernel
more	Display output one screen at a time
most	Browse or page through a text file
mount	Mount a file system
mtools	Manipulate MS-DOS files
mtr	Network diagnostics (traceroute/ping)
mv	Move or rename files or directories
mmv	Mass move and rename (files)
nc	Netcat, read and write data across networks
netstat	Networking connections/stats
nft	nftables for packet filtering and classification
nice	Set the priority of a command or job
nl	Number lines and write files
nohup	Run a command immune to hangups
notify-send	Send desktop notifications
nslookup	Query Internet name servers interactively
op	Operator access
open	Open a file in its default application
openssl	OpenSSL command line
passwd	Modify a user password
paste	Merge lines of files
pathchk	Check file name portability
Perf	Performance analysis tools for Linux
ping	Test a network connection
pgrep	List processes by name
pkill	Kill processes by name
popd •	Restore the previous value of the current directory
pr	Prepare files for printing
printcap	Printer capability database
printenv	Print environment variables
printf •	Format and print data
ps	Process status
pushd •	Save and then change the current directory
pv	Monitor the progress of data through a pipe
pwd •	Print working directory
quota	Display disk usage and limits
quotacheck	Scan a file system for disk usage
ram	ram disk device
rar	Archive files with compression
rcp	Copy files between two machines
read •	Read a line from standard input

Command	Description
`readarray` •	Read from STDIN into an array variable
`readonly` •	Mark variables/functions as readonly
`reboot`	Reboot the system
`rename`	Rename files
`renice`	Alter priority of running processes
`remsync`	Synchronize remote files via email
`return` •	Exit a shell function
`rev`	Reverse lines of a file
`rm`	Remove files
`rmdir`	Remove folder(s)
`rmmod`	Simple program to remove a module from the Linux Kernel
`rsync`	Remote file copy (synchronize file trees)
`screen`	Multiplex terminal, run remote shells via ssh
`scp`	Secure copy (remote file copy)
`sdiff`	Merge two files interactively
`sed`	Stream editor
`select`	Accept user choices via keyboard input
`seq`	Print numeric sequences
`set` •	Manipulate shell variables and functions
`setfacl`	Set file access control lists.
`sftp`	Secure file transfer program
`sha256sum`	Compute and check SHA256 (256-bit) checksums
`shift` •	Shift positional parameters
`shopt` •	Shell options
`shuf`	Generate random permutations
`shutdown`	Shutdown or restart Linux
`sleep`	Delay for a specified time
`slocate`	Find files
`sntp`	Simple Network Time Protocol client program
`sort`	Sort text files
`source` •	Run commands from a file '.'
`split`	Split a file into fixed-size pieces
`ss`	Socket statistics
`ssh`	Secure shell client (remote login program)
`stat`	Display file or file system status
`strace`	Trace system calls and signals
`su`	Substitute user identity
`sudo`	Execute a command as another user
`sum`	Print a checksum for a file
`suspend` •	Suspend execution of this shell
`sync`	Synchronize data on disk with memory
`tabs`	Set tabs on a terminal
`tac`	Concatenate and write files in reverse
`tail`	Output the last part of a file
`tar`	Store, list or extract files in an archive
`tee`	Redirect output to multiple files
`test` •	Evaluate a conditional expression
`time`	Measure program running time
`timeout`	Run a command with a time limit
`times` •	User and system times

Command	Description
tmux	Terminal multiplexer
touch	Change file timestamps
top	List processes running on the system
tput	Set terminal-dependent capabilities, colour, position
traceroute	Trace route to host
trap •	Execute a command when the shell receives a signal
tr	Translate, squeeze, and/or delete characters
true •	Do nothing, successfully
tsort	Topological sort
tty	Print filename of terminal on stdin
type •	Describe a command
ulimit •	Limit user resources
umask •	Users file creation mask
umount	Unmount a device
unalias •	Remove an alias
uname	Print system information
unexpand	Convert spaces to tabs
uniq	Uniquify files
units	Convert units from one scale to another
unix2dos	UNIX to Windows or MAC text file format converter
unrar	Extract files from a rar archive
unset •	Remove variable or function names
unshar	Unpack shell archive scripts
until	Execute commands (until error)
uptime	Show uptime
useradd	Create new user account
userdel	Delete a user account
usermod	Modify user account
users	List users currently logged in
uuencode	Encode a binary file
uudecode	Decode a file created by uuencode
v	Verbosely list directory contents ('ls -l -b')
vdir	Verbosely list directory contents ('ls -l -b')
vi	Text editor
vmstat	Report virtual memory statistics
w	Show who is logged on and what they are doing
wait •	Wait for a process to complete
watch	Execute/display a program periodically
wc	Print byte, word, and line counts
whereis	Search the user's $path, man pages and source files for a program
which	Search the user's $path for a program file
while	Execute commands
who	Print all usernames currently logged in
whoami	Print the current user id and name ('id -un')
wget	Retrieve web pages or files via HTTP, HTTPS or FTP
wl-copy	Copy to clipboard (Wayland)
wl-paste	Paste from clipboard (Wayland)
write	Send a message to another user
xargs	Execute utility, passing constructed argument list(s)
xclip	Copy to clipboard (X11)

Command	Description
`xdg-open`	Open a file or URL in the user's preferred application.
`xxd`	Make a hexdump or do the reverse
`xz`	Compress or decompress .xz and .lzma files
`yes`	Print a string until interrupted
`zip`	Package and compress (archive) files
`.`	Run a command script in the current shell
`!!`	Run the last command again
`#`	Comment / remark